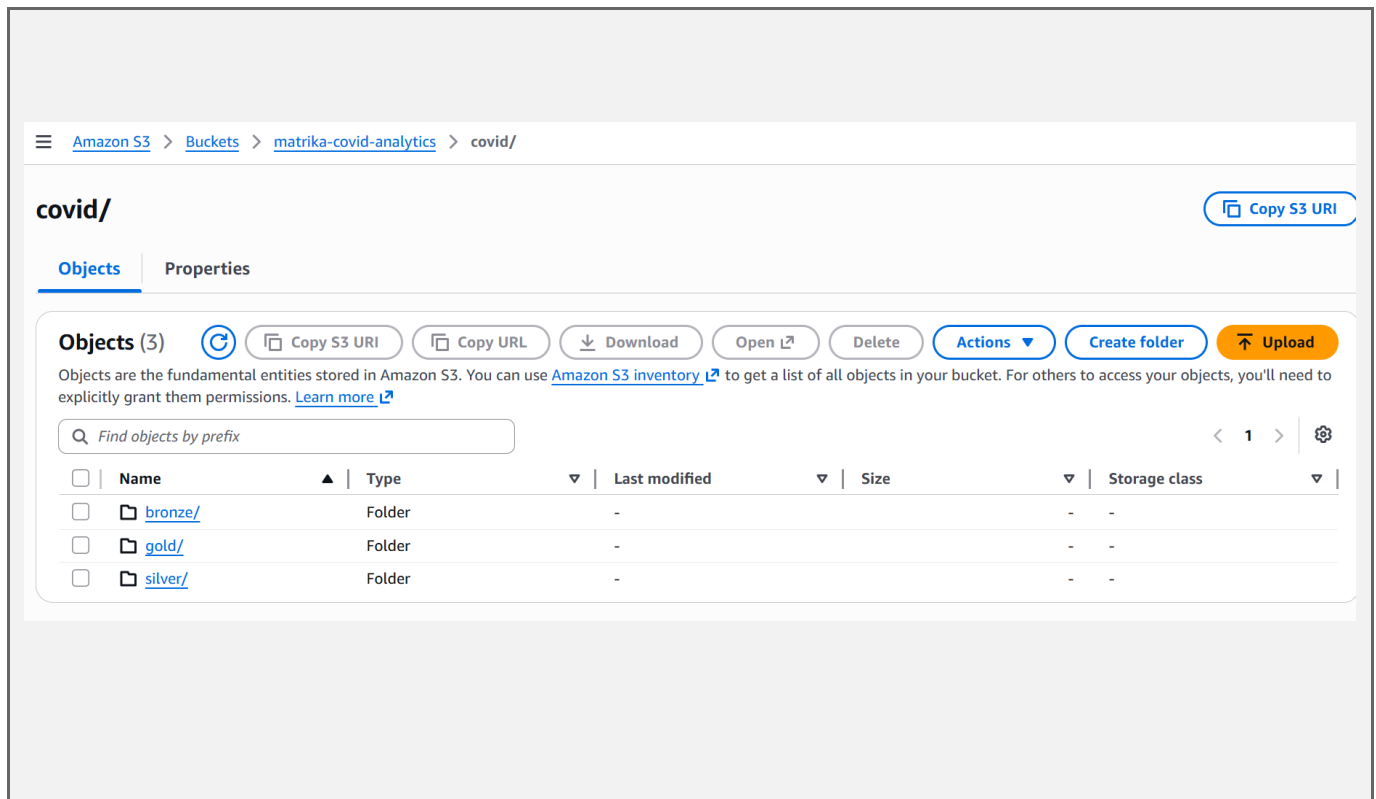


COVID-19 Analytics on AWS

End-to-End Data Engineering Project

Student Name	Matrika Subedi
Course / Assignment	COVID-19 Analytics (AWS)
Submission Date	5/29/2026
AWS Region	us-east-2
S3 Bucket	s3://matrika-covid-analytics/covid/

Screenshot 1: Final S3 folder structure showing bronze, silver, and gold folders



1. Project Overview

This project builds an end-to-end COVID-19 analytics pipeline on AWS. Raw CSV data is stored in Amazon S3 Bronze, transformed into standardized Parquet files in Silver using AWS Glue PySpark, modeled into Gold star-schema tables, validated with Athena, and loaded into Amazon Redshift for analytics.

Architecture Flow

Bronze CSV data in S3 → Glue PySpark standardization → Silver Parquet → Glue star schema build → Gold Parquet → Athena validation → Redshift COPY → BI SQL queries

Screenshot 2: Architecture diagram or AWS services overview

[Insert screenshot here]

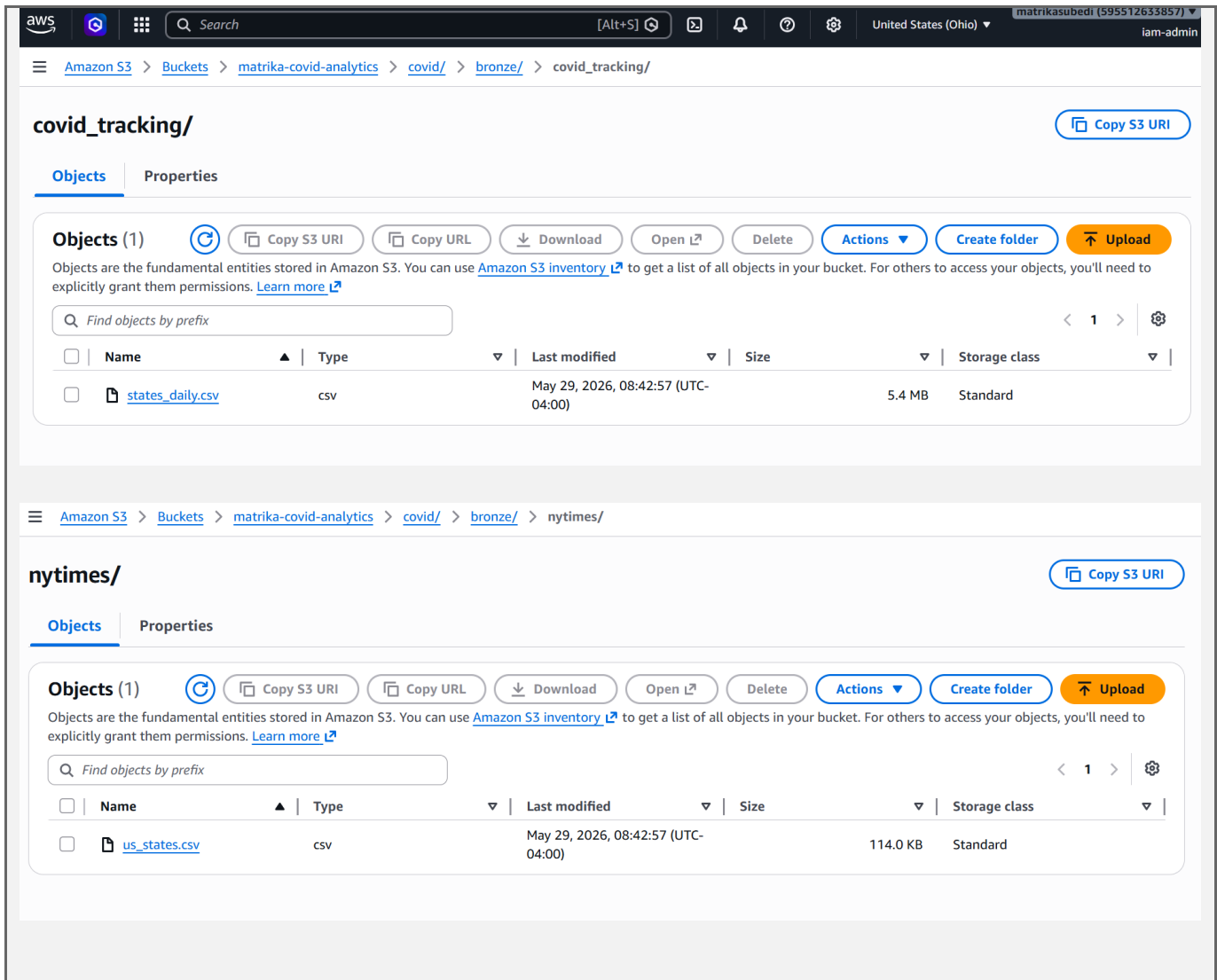
Status	Task
[ddone]	Created S3 bucket and folder structure
[done]	Uploaded raw CSV files into Bronze layer
[done]	Created and ran Bronze-to-Silver Glue job
[done]	Created Silver external tables in Athena
[done]	Created and ran Silver-to-Gold Glue job
[done]	Created Gold external tables in Athena
[done]	Created Redshift schema and internal tables
[done]	Copied Gold data into Redshift
[done]	Ran BI analytics SQL queries

2. S3 Data Lake Structure

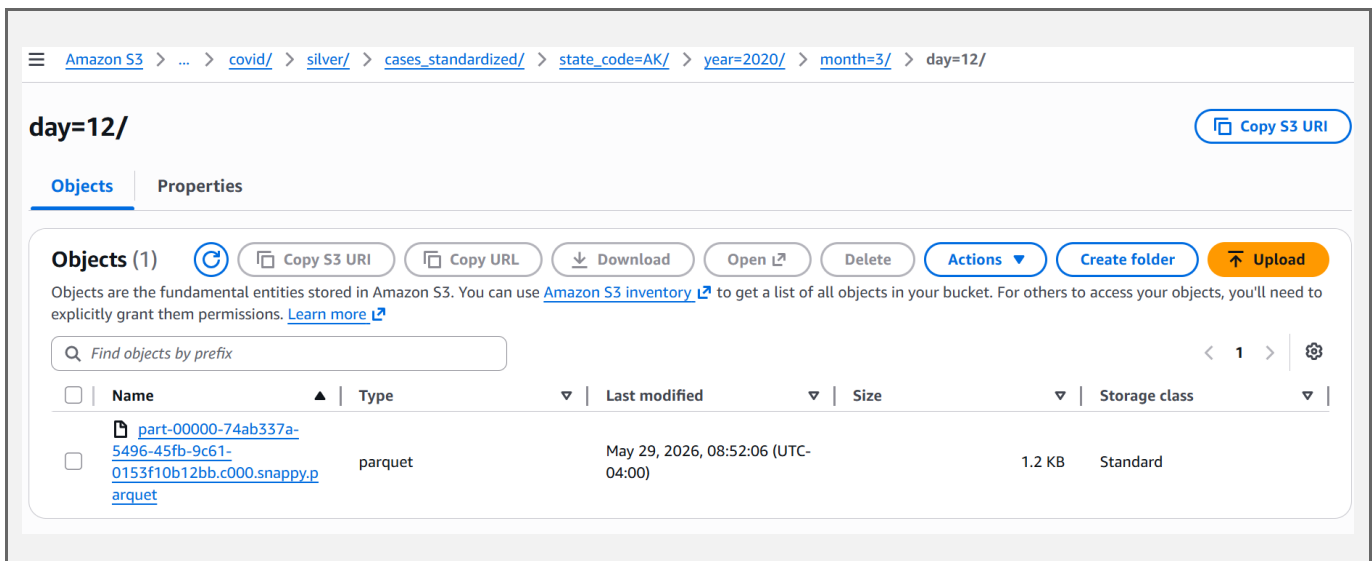
The project uses a three-layer S3 layout: Bronze for raw source data, Silver for standardized Parquet, and Gold for analytics-ready star schema data.

```
s3://matrika-covid-analytics/covid/  
  bronze/  
    nytimes/us_states.csv  
    covid_tracking/states_daily.csv  
    static/states_abv.csv  
  silver/  
    cases_standardized/  
    testing_standardized/  
  gold/  
    dim_date/  
    dim_state/  
    fact_cases_state_daily/  
    fact_testing_state_daily/
```

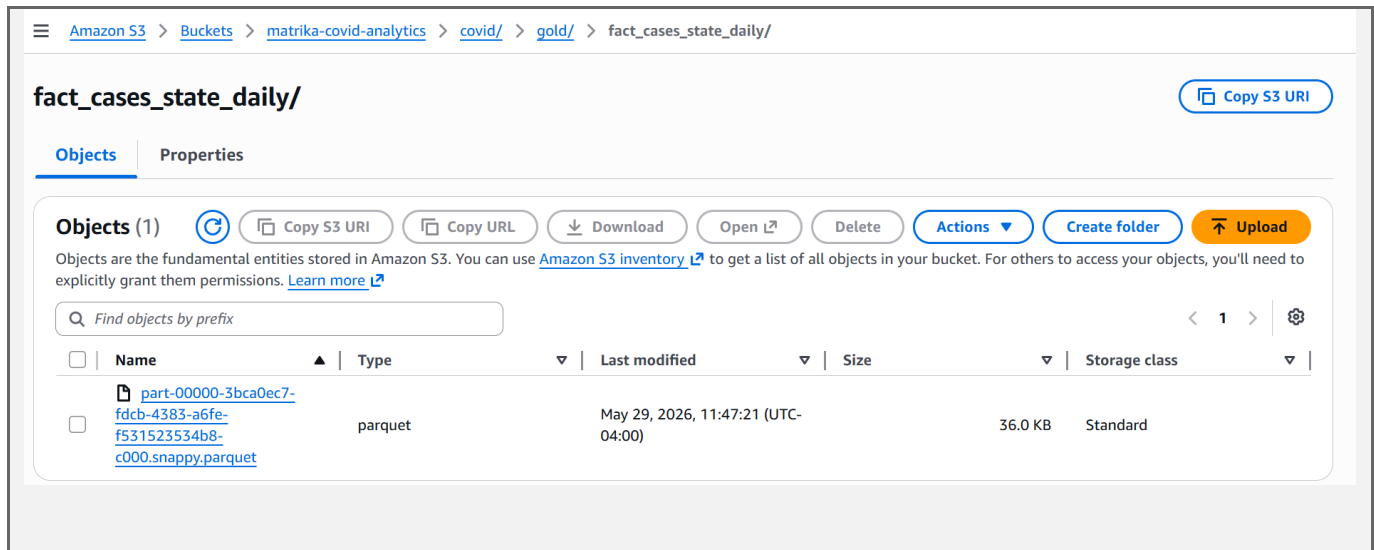
Screenshot 3: Bronze layer files in S3



Screenshot 4: Silver layer Parquet output in S3



Screenshot 5: Gold layer Parquet output in S3



3. Bronze to Silver Transformation

The Bronze-to-Silver Glue job reads raw CSV files, standardizes dates and state names/codes, casts numeric columns, and writes partitioned Parquet files to the Silver layer.

3.1 Bronze-to-Silver PySpark Code

```
from pyspark.sql import SparkSession, functions as F

BUCKET = "matrika-covid-analytics"
BRONZE = f"s3://{BUCKET}/covid/bronze"
SILVER = f"s3://{BUCKET}/covid/silver"

spark = SparkSession.builder.appName("covid-minimal-silver").getOrCreate()

upper_trim = lambda c: F.upper(F.trim(F.col(c)))

states = (
    spark.read.option("header", True)
    .csv(f"{BRONZE}/static/states_abv.csv")
    .select(
        upper_trim("Abbreviation").alias("state_code"),
        F.initcap("State").alias("state_name")
    )
)

cases_raw = spark.read.option("header", True).csv(f"{BRONZE}/nytimes/us_states.csv")

cases_std = (
    cases_raw
    .withColumn("full_date", F.to_date("date"))
```

```

.withColumn("state_name_raw", F.initcap(F.col("state")))
.join(states, states.state_name == F.col("state_name_raw"), "left")
.withColumn("cases_cum", F.col("cases").cast("long"))
.withColumn("deaths_cum", F.col("deaths").cast("long"))
.withColumn("year", F.year("full_date"))
.withColumn("month", F.month("full_date"))
.withColumn("day", F.dayofmonth("full_date"))
.select(
    "full_date",
    "state_code",
    "state_name",
    "cases_cum",
    "deaths_cum",
    "year",
    "month",
    "day"
)
.dropna(subset=["full_date", "state_code"])
)

cases_std.write.mode("overwrite") \
    .partitionBy("state_code", "year", "month", "day") \
    .parquet(f"{SILVER}/cases_standardized")

tests_raw = spark.read.option("header", True).csv(f"{BRONZE}/covid_tracking/states_daily.csv")

tests_std = (
    tests_raw
    .withColumn("full_date", F.to_date(F.col("date").cast("string"), "yyyyMMdd"))
    .withColumn("state_code", upper_trim("state"))
    .join(states.select("state_code", "state_name"), "state_code", "left")
    .withColumn("tests_total_cum", F.col("totalTestResults").cast("long"))
    .withColumn("tests_pos_cum", F.col("positive").cast("long"))
    .withColumn("tests_neg_cum", F.col("negative").cast("long"))
    .withColumn("year", F.year("full_date"))
    .withColumn("month", F.month("full_date"))
    .withColumn("day", F.dayofmonth("full_date"))
    .select(
        "full_date",
        "state_code",
        "state_name",
        "tests_total_cum",
        "tests_pos_cum",

```

```

        "tests_neg_cum",
        "year",
        "month",
        "day"
    )
    .dropna(subset=["full_date", "state_code"])
)

tests_std.write.mode("overwrite") \
    .partitionBy("state_code", "year", "month", "day") \
    .parquet(f"{SILVER}/testing_standardized")

print("Silver complete.")

```

Screenshot 6: Successful Bronze-to-Silver Glue job run

The screenshot displays the AWS Glue console interface. On the left, the navigation pane shows 'AWS Glue' with options like 'Getting started', 'ETL jobs', 'Visual ETL', 'Notebooks', 'Job run monitoring', 'Data Catalog tables', 'Data connections', 'Workflows (orchestration)', and 'Zero-ETL integrations'. The 'Data Catalog' section is expanded, showing 'Catalogs' and 'Databases'. The main content area shows the details for a job named 'bronze_to_silver.py', which was last modified on 5/29/2026 at 8:50:50 AM. The 'Runs' tab is selected, showing a table of job runs. The first run is listed as 'Succeeded' with 0 retries, starting at 05/29/2026 08:51:04 and ending at 05/29/2026 09:15:12, with a duration of 24 m 1 s and a capacity of 10 DPU's. The interface includes buttons for 'View details', 'Stop job run', and 'Troubleshoot with AI'.

Run status	Retries	Start time (Local)	End time (Local)	Duration	Capacit...
Succeeded	0	05/29/2026 08:51:04	05/29/2026 09:15:12	24 m 1 s	10 DPU's

5. Silver to Gold Star Schema Transformation

The Silver-to-Gold Glue job creates dimension and fact tables. The fact tables are written without partitioning so Redshift COPY can load all columns directly from the Parquet files.

5.1 Silver-to-Gold PySpark Code

```

from pyspark.sql import SparkSession, functions as F, Window as W

BUCKET = "matrika-covid-analytics"
SILVER = f"s3://{BUCKET}/covid/silver"
GOLD = f"s3://{BUCKET}/covid/gold"

```

```

spark = SparkSession.builder.appName("covid-minimal-gold").getOrCreate()

cases = spark.read.parquet(f"{SILVER}/cases_standardized")
tests = spark.read.parquet(f"{SILVER}/testing_standardized")

# dim_date
all_dates = (
    cases.select(F.to_date("full_date").alias("full_date"))
    .union(tests.select(F.to_date("full_date").alias("full_date")))
    .dropDuplicates()
)

dim_date = (
    all_dates
    .withColumn("date_id", F.date_format("full_date", "yyyyMMdd").cast("int"))
    .withColumn("year", F.year("full_date").cast("int"))
    .withColumn("month", F.month("full_date").cast("int"))
    .withColumn("day", F.dayofmonth("full_date").cast("int"))
    .withColumn("dow", F.dayofweek("full_date").cast("int"))
    .withColumn("is_weekend", F.col("dow").isin([1, 7]))
    .select(
        "date_id",
        "full_date",
        "year",
        "month",
        "day",
        "dow",
        "is_weekend"
    )
)

dim_date.write.mode("overwrite").parquet(f"{GOLD}/dim_date")

dim_state = (
    cases.select("state_code", "state_name")
    .dropDuplicates()
)

dim_state.write.mode("overwrite").parquet(f"{GOLD}/dim_state")

w = W.partitionBy("state_code").orderBy("full_date")

fact_cases = (

```

```

cases
.withColumn("date_id", F.date_format("full_date", "yyyyMMdd").cast("int"))
.withColumn(
    "new_cases",
    F.greatest(
        F.col("cases_cum") - F.lag("cases_cum").over(w),
        F.lit(0)
    )
)
.withColumn(
    "new_deaths",
    F.greatest(
        F.col("deaths_cum") - F.lag("deaths_cum").over(w),
        F.lit(0)
    )
)
.select(
    "date_id",
    "state_code",
    "cases_cum",
    "deaths_cum",
    "new_cases",
    "new_deaths"
)
)

```

```

fact_cases.write.mode("overwrite") \
    .parquet(f"{GOLD}/fact_cases_state_daily")

```

```

w2 = W.partitionBy("state_code").orderBy("full_date")

```

```

fact_tests = (
    tests
.withColumn("date_id", F.date_format("full_date", "yyyyMMdd").cast("int"))
.withColumn(
    "new_tests",
    F.greatest(
        F.col("tests_total_cum") - F.lag("tests_total_cum").over(w2),
        F.lit(0)
    )
)
.withColumn(
    "positivity_rate",

```



```

        F.when(
            F.col("tests_total_cum") > 0,
            (F.col("tests_pos_cum") / F.col("tests_total_cum")).cast("double")
        )
    )
)

.select(
    "date_id",
    "state_code",
    "tests_total_cum",
    "tests_pos_cum",
    "tests_neg_cum",
    "new_tests",
    "positivity_rate"
)

)

fact_tests.write.mode("overwrite") \
    .parquet(f"{GOLD}/fact_testing_state_daily")

print("Gold complete.")

```

Screenshot 9: Successful Silver-to-Gold Glue job run

The screenshot shows the AWS Glue console interface. On the left is a navigation menu with 'AWS Glue' and 'Data Catalog' sections. The main panel displays the 'silver_to_gold.py' job details. The 'Runs' tab is active, showing a table of job runs. The first run is highlighted, showing a 'Succeeded' status.

Run status	Retries	Start time (Local)	End time (Local)	Duration	Capacit...
✔ Succeeded	0	05/29/2026 11:45:37	05/29/2026 11:47:48	2 m 5 s	10 DPU
✔ Succeeded	0	05/29/2026 11:06:26	05/29/2026 11:08:47	2 m 9 s	10 DPU

Screenshot 10: Gold S3 folders with non-partitioned fact Parquet files



Amazon S3 > Buckets > matrika-covid-analytics > covid/ > gold/ > fact_cases_state_daily/

fact_cases_state_daily/ [Copy S3 URI](#)

Objects | Properties

Objects (1) [Copy S3 URI](#) [Copy URL](#) [Download](#) [Open](#) [Delete](#) [Actions](#) [Create folder](#) [Upload](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	part-00000-3bca0ec7-fdcb-4383-a6fe-f531523534b8-c000.snappy.parquet	parquet	May 29, 2026, 11:47:21 (UTC-04:00)	36.0 KB	Standard

6. Athena Gold Tables and Validation

```
CREATE DATABASE IF NOT EXISTS covid_gold_db;

CREATE EXTERNAL TABLE IF NOT EXISTS covid_gold_db.dim_date (
  date_id int,
  full_date date,
  year int,
  month int,
  day int,
  dow int,
  is_weekend boolean
)
STORED AS PARQUET
LOCATION 's3://matrika-covid-analytics/covid/gold/dim_date/';

CREATE EXTERNAL TABLE IF NOT EXISTS covid_gold_db.dim_state (
  state_code string,
  state_name string
)
STORED AS PARQUET
LOCATION 's3://matrika-covid-analytics/covid/gold/dim_state/';

CREATE EXTERNAL TABLE IF NOT EXISTS covid_gold_db.fact_cases_state_daily (
  date_id int,
  state_code string,
  cases_cum bigint,
  deaths_cum bigint,
  new_cases bigint,
  new_deaths bigint
)
STORED AS PARQUET
LOCATION 's3://matrika-covid-analytics/covid/gold/fact_cases_state_daily/';

CREATE EXTERNAL TABLE IF NOT EXISTS covid_gold_db.fact_testing_state_daily (
  date_id int,
  state_code string,
  tests_total_cum bigint,
  tests_pos_cum bigint,
  tests_neg_cum bigint,
  new_tests bigint,
  positivity_rate double
)
```

```

STORED AS PARQUET
LOCATION 's3://matrika-covid-analytics/covid/gold/fact_testing_state_daily/';

SELECT COUNT(*) FROM covid_gold_db.dim_date; 420
SELECT COUNT(*) FROM covid_gold_db.dim_state; 51
SELECT COUNT(*) FROM covid_gold_db.fact_cases_state_daily; 3540
SELECT COUNT(*) FROM covid_gold_db.fact_testing_state_daily; 20780

```

Screenshot 11: Athena Gold tables and row counts

The screenshot displays the Amazon Athena Query Editor interface. On the left, the 'Tables and views' sidebar lists four tables: `dim_date`, `dim_state`, `fact_cases_state_daily`, and `fact_testing_state_daily`. The main editor shows a SQL query with three lines, where the third line is selected: `SELECT COUNT(*) FROM covid_gold_db.fact_testing_state_daily;`. Below the query, the 'Run again' button is highlighted. The 'Query results' tab is active, showing a green 'Completed' status bar with 'Time in queue: 109 ms' and 'Run time: 360 ms'. Below this, the 'Results (1)' section shows a single row with the value '20780'.

SQL Ln 50, Col 61

Run Explain Cancel Clear Create

Query results Query status

Completed Time in queue: 109 ms Run time: 360 ms Data scanned: -

Results (1) Copy Download results CSV

Search rows

#	_col0
1	20780

7. Redshift Serving Layer

The Gold Parquet data is copied into Redshift internal tables for analytics. Redshift COPY from Parquet is strict about column order and data types, so the Glue output and Redshift DDL are aligned.

7.1 Redshift DDL

```
CREATE SCHEMA IF NOT EXISTS covid_gold;

DROP TABLE IF EXISTS covid_gold.fact_testing_state_daily;
DROP TABLE IF EXISTS covid_gold.fact_cases_state_daily;
DROP TABLE IF EXISTS covid_gold.dim_state;
DROP TABLE IF EXISTS covid_gold.dim_date;

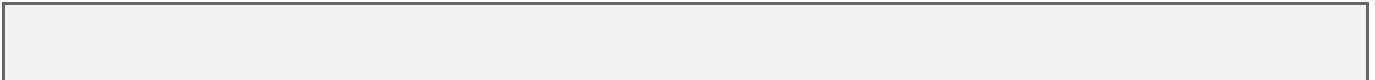
CREATE TABLE covid_gold.dim_date (
    date_id int,
    full_date date,
    year int,
    month int,
    day int,
    dow int,
    is_weekend boolean
)
SORTKEY(date_id);

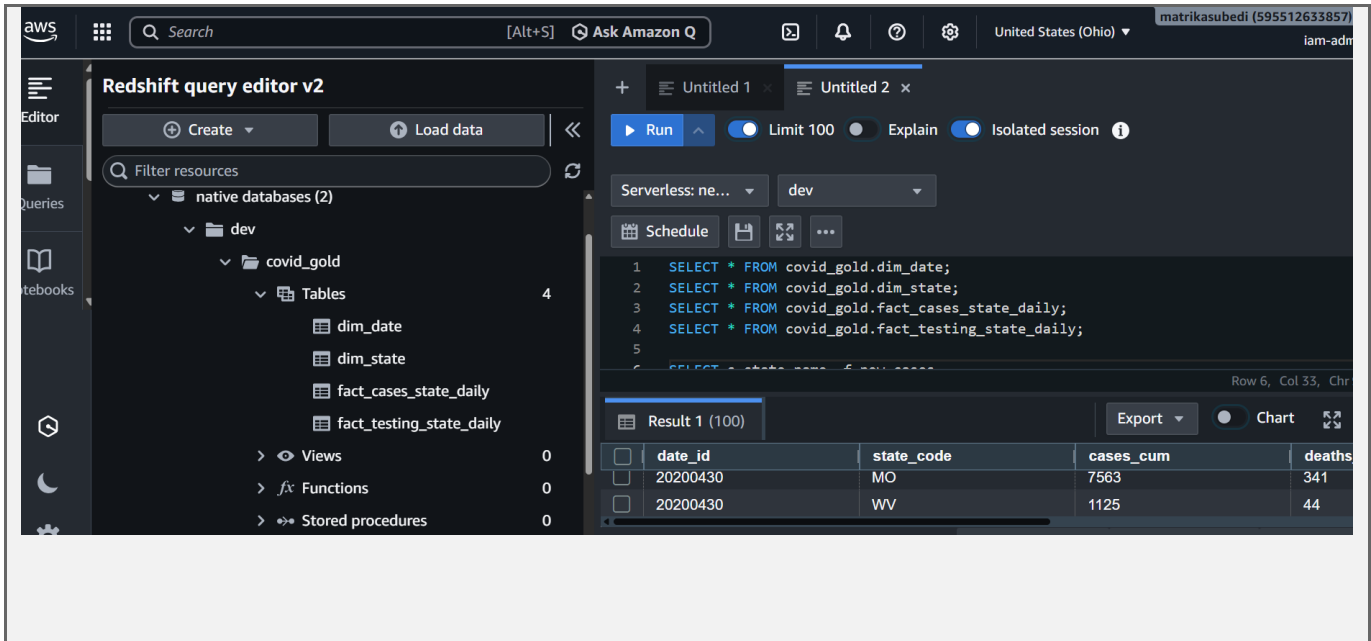
CREATE TABLE covid_gold.dim_state (
    state_code varchar(2),
    state_name varchar(64)
)
DISTSTYLE ALL;

CREATE TABLE covid_gold.fact_cases_state_daily (
    date_id int NOT NULL,
    state_code varchar(2) NOT NULL,
    cases_cum bigint,
    deaths_cum bigint,
    new_cases bigint,
    new_deaths bigint
)
DISTKEY(state_code)
SORTKEY(date_id, state_code);

CREATE TABLE covid_gold.fact_testing_state_daily (
    date_id int NOT NULL,
    state_code varchar(2) NOT NULL,
    tests_total_cum bigint,
    tests_pos_cum bigint,
    tests_neg_cum bigint,
    new_tests bigint,
    positivity_rate double precision
)
DISTKEY(state_code)
SORTKEY(date_id, state_code);
```

Screenshot 12: Redshift tables created successfully





7.2 Redshift COPY Commands

```
COPY covid_gold.dim_date
FROM 's3://matrika-covid-analytics/covid/gold/dim_date/'
IAM_ROLE 'arn:aws:iam::<ACCOUNT_ID>:role/<RedshiftCopyRole>'
FORMAT AS PARQUET;

COPY covid_gold.dim_state
FROM 's3://matrika-covid-analytics/covid/gold/dim_state/'
IAM_ROLE 'arn:aws:iam::<ACCOUNT_ID>:role/<RedshiftCopyRole>'
FORMAT AS PARQUET;

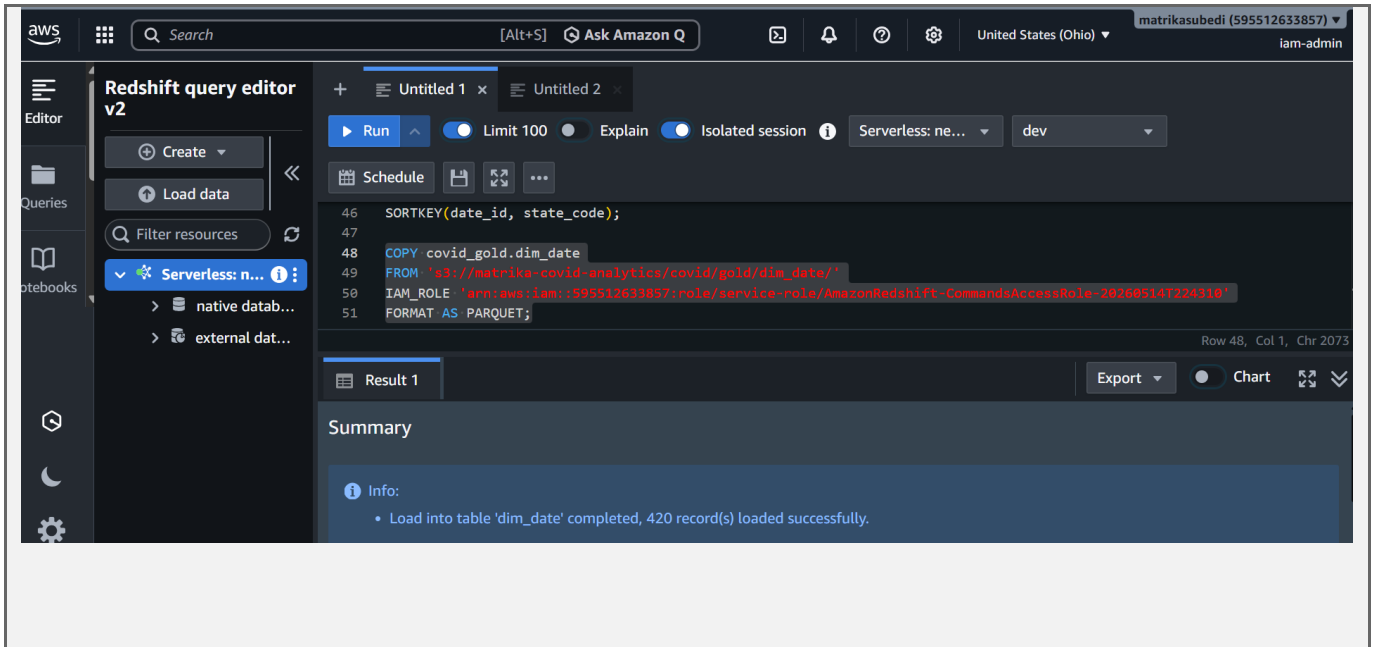
COPY covid_gold.fact_cases_state_daily
FROM 's3://matrika-covid-analytics/covid/gold/fact_cases_state_daily/'
IAM_ROLE 'arn:aws:iam::<ACCOUNT_ID>:role/<RedshiftCopyRole>'
FORMAT AS PARQUET;

COPY covid_gold.fact_testing_state_daily
FROM 's3://matrika-covid-analytics/covid/gold/fact_testing_state_daily/'
IAM_ROLE 'arn:aws:iam::<ACCOUNT_ID>:role/<RedshiftCopyRole>'
FORMAT AS PARQUET;

ANALYZE VERBOSE;
```

Screenshot 13: Successful Redshift COPY command results





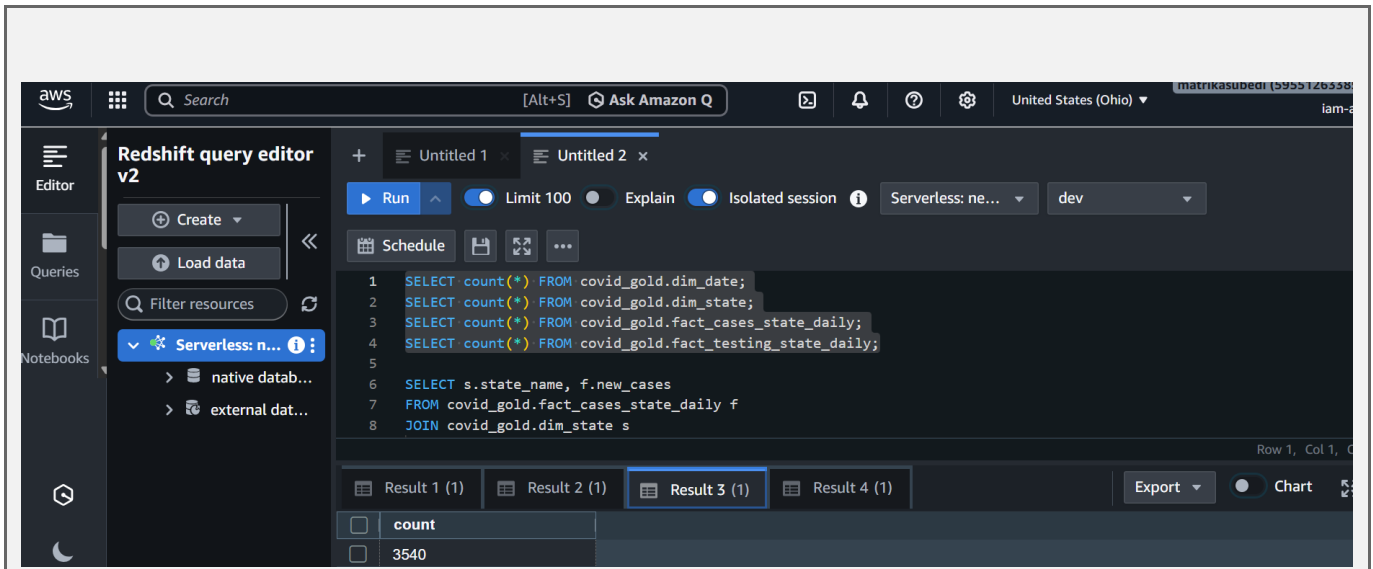
7.3 Redshift Data Validation

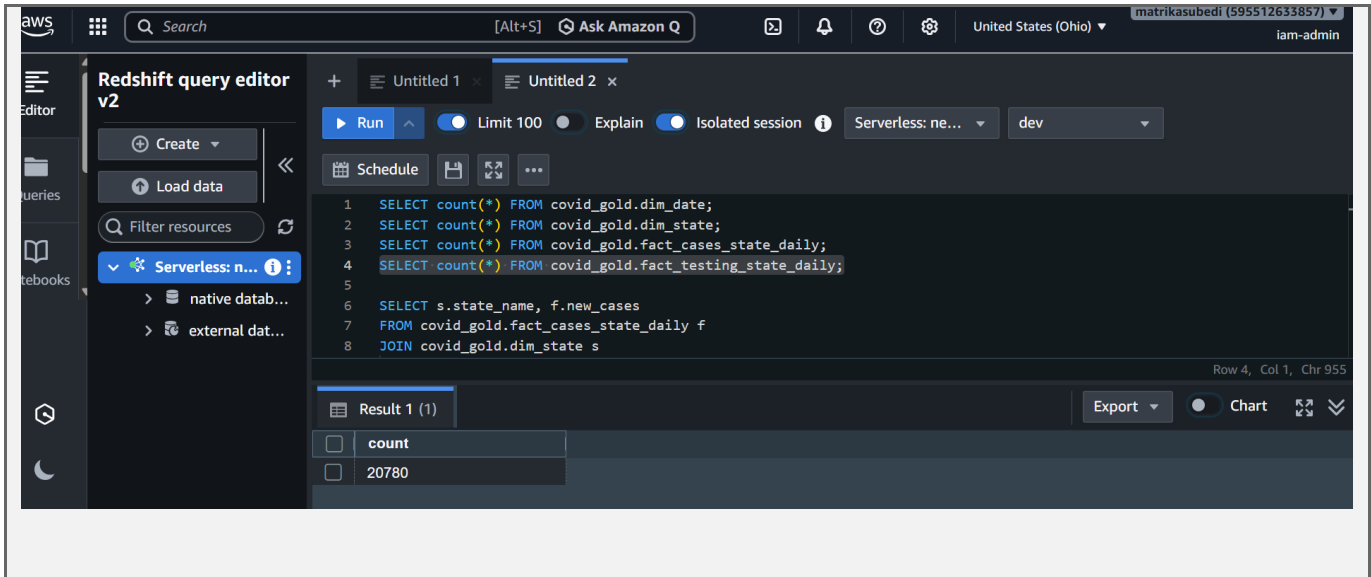
```

SELECT COUNT(*) FROM covid_gold.dim_date;      420
SELECT COUNT(*) FROM covid_gold.dim_state;     51
SELECT COUNT(*) FROM covid_gold.fact_cases_state_daily;  3540
SELECT COUNT(*) FROM covid_gold.fact_testing_state_daily; 20780

```

Screenshot 14: Redshift row counts and relationship validation





8. BI Analytics Queries

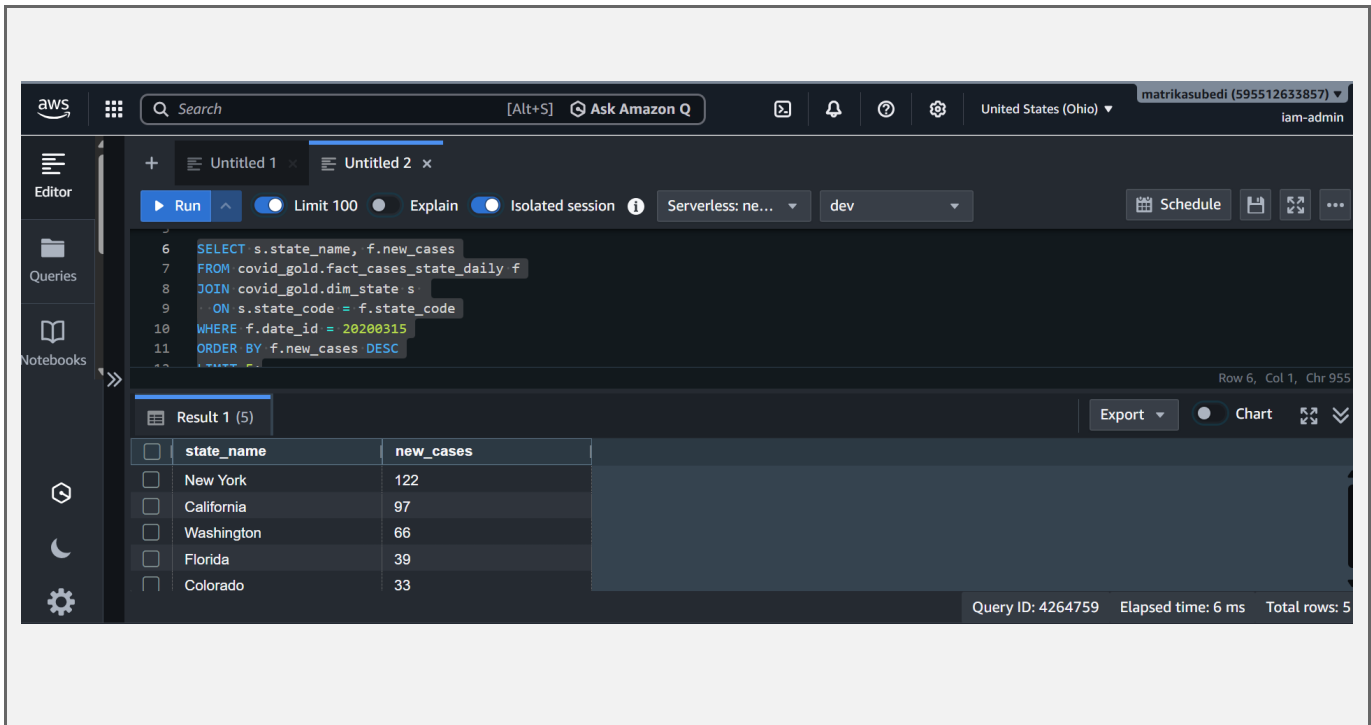
8.1 Top 5 states by new cases on a selected date

```

SELECT s.state_name, f.new_cases
FROM covid_gold.fact_cases_state_daily f
JOIN covid_gold.dim_state s
  ON s.state_code = f.state_code
WHERE f.date_id = 20200515
ORDER BY f.new_cases DESC
LIMIT 5;

```

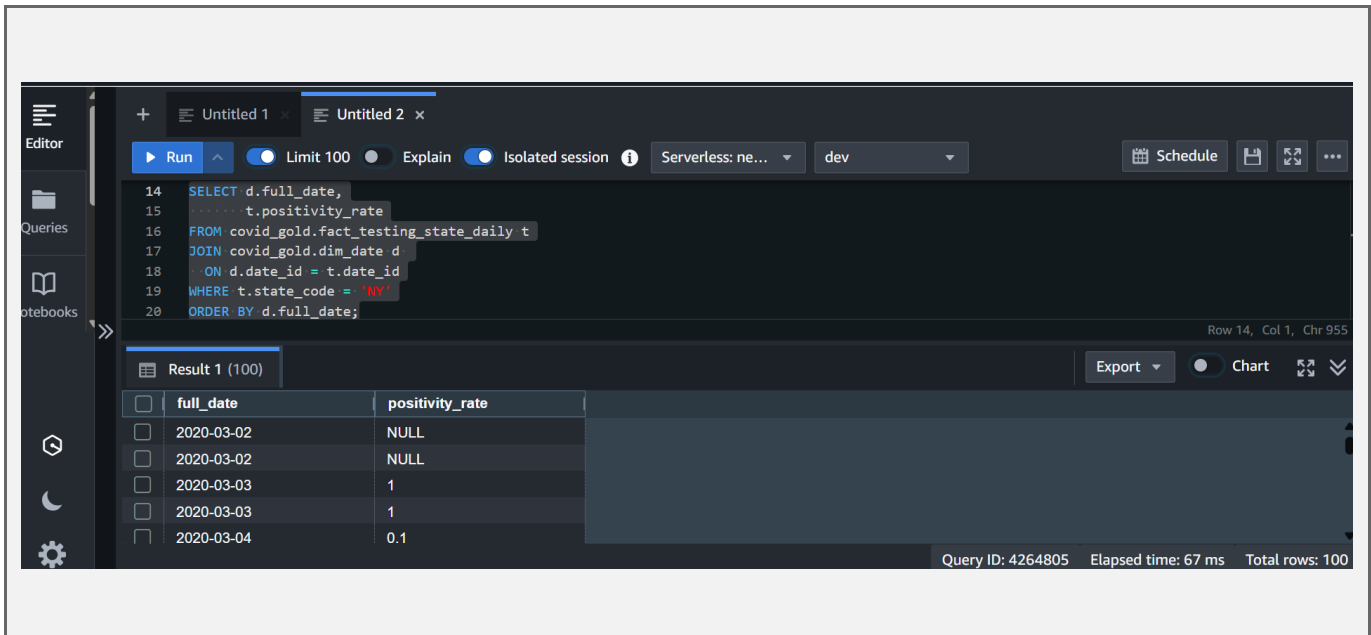
Screenshot 15: Top 5 states by new cases query result



8.2 Positivity rate trend for New York

```
SELECT d.full_date,
       t.positivity_rate
FROM covid_gold.fact_testing_state_daily t
JOIN covid_gold.dim_date d
     ON d.date_id = t.date_id
WHERE t.state_code = 'NY'
ORDER BY d.full_date;
```

Screenshot 16: Positivity rate trend result



The screenshot shows a SQL query editor with the following query:

```
14 SELECT d.full_date,
15        t.positivity_rate
16 FROM covid_gold.fact_testing_state_daily t
17 JOIN covid_gold.dim_date d
18      ON d.date_id = t.date_id
19 WHERE t.state_code = 'NY'
20 ORDER BY d.full_date;
```

The query is executed, and the results are displayed in a table. The table has two columns: **full_date** and **positivity_rate**. The results are as follows:

full_date	positivity_rate
2020-03-02	NULL
2020-03-02	NULL
2020-03-03	1
2020-03-03	1
2020-03-04	0.1

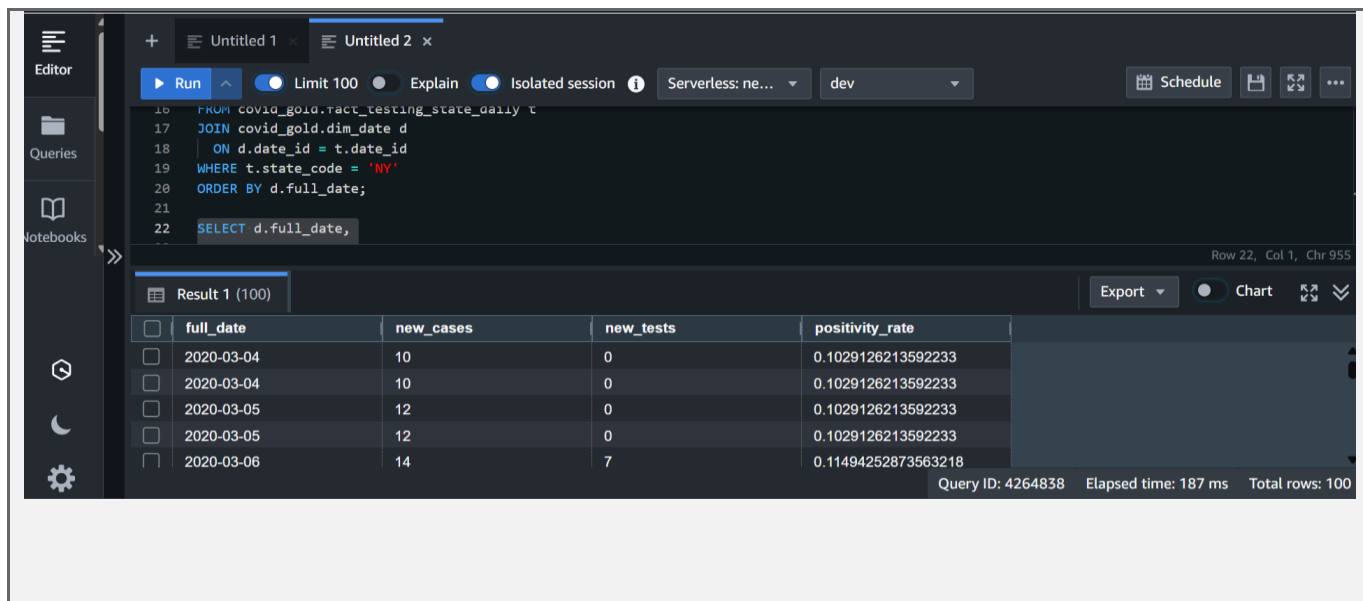
The interface also shows a sidebar with 'Editor', 'Queries', and 'Notebooks' tabs. The 'Editor' tab is active, and the query is being edited. The 'Run' button is visible, and the 'Limit 100' option is selected. The 'Serverless: ne...' and 'dev' environment settings are shown. The 'Schedule' button is also present. The 'Export' and 'Chart' buttons are visible in the top right corner of the results area. The status bar at the bottom indicates 'Query ID: 4264805', 'Elapsed time: 67 ms', and 'Total rows: 100'.

8.3 Daily cases vs tests for California

```
SELECT d.full_date,
       c.new_cases,
       t.new_tests,
       t.positivity_rate
FROM covid_gold.fact_cases_state_daily c
JOIN covid_gold.fact_testing_state_daily t
     ON t.date_id = c.date_id
     AND t.state_code = c.state_code
JOIN covid_gold.dim_date d
     ON d.date_id = c.date_id
WHERE c.state_code = 'CA'
ORDER BY d.full_date;
```

Screenshot 17: Daily cases vs tests query result





9. Issues Faced and Resolutions

Issue	Cause	Resolution
S3 NoSuchKey for testing data	Opened S3 prefix instead of actual object key	Used the exact states_daily.csv S3 key and uploaded it into Bronze.
PATH_NOT_FOUND for states_abv.csv	Lookup CSV was missing from Bronze static folder	Uploaded states_abv.csv to covid/bronze/static/.
Unresolved column abbr	CSV headers were State and Abbreviation, but code expected abbr and name	Updated PySpark lookup code to use State and Abbreviation.
Spark date format error for u pattern	Glue/Spark 3+ does not support date_format pattern u	Replaced date_format(full_date, "u") with dayofweek(full_date).
Redshift Parquet schema mismatch	Redshift DDL used smallint/int while Parquet had int/bigint	Aligned Redshift DDL types with Spark Parquet output.
Redshift unmatched columns in fact table	Fact files were partitioned by state_code, so state_code was not inside each Parquet file	Removed partitionBy("state_code") for Gold fact writes used by direct Redshift COPY.
dim_date date_id mismatch	Parquet column order did not match Redshift table order	Added explicit select order in dim_date before writing Parquet.

10. Final Summary

The project successfully demonstrates an end-to-end AWS analytics pipeline. Raw COVID-19 CSV data was stored in S3 Bronze, standardized into Silver Parquet using Glue PySpark, transformed into Gold dimension and fact tables, validated in Athena, and loaded into Redshift for SQL analytics.

Submission Evidence Checklist

Status	Task
[done]	S3 Bronze screenshot added
[done]	S3 Silver screenshot added
[done]	S3 Gold screenshot added
[done]	Glue Bronze-to-Silver success screenshot added
[done]	Glue Silver-to-Gold success screenshot added
[done]	Athena Silver validation screenshot added
[done]	Athena Gold validation screenshot added

[done]	Redshift COPY success screenshot added
---------	--